Quellcode mit dem `listings`-Paket darstellen

Fortbildungsmaterial für MINT-Lehrkräfte

25. Mai 2025

Das `listings`-Paket ist ein unverzichtbares Werkzeug für MINT-Lehrkräfte, die Quellcode in ihren Unterrichtsmaterialien präsentieren möchten. Es ermöglicht die elegante und professionelle Darstellung von Programmcode verschiedenster Sprachen mit automatischer Syntaxhervorhebung, Zeilennummerierung und flexibler Formatierung. Diese Handreichung zeigt, wie Sie mit wenigen Befehlen ansprechende Code-Darstellungen erstellen und durch eigene Stilvorlagen konsistente Formatierungen für Ihre Materialien entwickeln können.

1 Grundlagen des `listings`-Pakets

Das `listings`-Paket wird in der Präambel Ihres LaTeX-Dokuments eingebunden und bietet sofort einsatzbereite Funktionalität für die Code-Darstellung:

```
\usepackage{listings}
\usepackage{xcolor}  % fuer Farbunterstuetzung
```

Wichtig zu wissen ist, dass das `listings`-Paket nicht für alle existierenden Programmiersprachen vordefinierte Syntaxhervorhebungen mitbringt. Zu den vollständig unterstützten Sprachen gehören unter anderem Python, Java, C/C++, HTML, SQL, PHP, MATLAB, Pascal und TeX (für LaTeX). Für weniger verbreitete oder neuere Sprachen wie CSS, JavaScript, TypeScript oder moderne Frameworks müssen Sie möglicherweise eigene Sprachdefinitionen erstellen oder auf die allgemeine Formatierung ohne spezifische Syntaxhervorhebung zurückgreifen. Eine vollständige Liste der unterstützten Sprachen finden Sie in der Paketdokumentation unter `texdoc listings`. Falls eine benötigte Sprache nicht verfügbar ist, können Sie mit `\lstdefinelanguage` eigene Definitionen erstellen oder das Code-Beispiel ohne Sprachangabe verwenden, wodurch immerhin die grundlegende Formatierung (Schriftart, Rahmen, etc.) erhalten bleibt.
Der einfachste Weg, Code darzustellen, ist die Verwendung der `lstlisting`-Umgebung:

```
\begin{lstlisting}[language=Python]
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10))
\end{lstlisting}
```

Ausgabe

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(10))
```

Die Angabe der Programmiersprache erfolgt über den Parameter `language=`, wodurch automatisch die entsprechende Syntaxhervorhebung aktiviert wird. Das Paket erkennt Schlüsselwörter, Kommentare, Strings und andere Sprachelemente und formatiert sie entsprechend.

2 Eigene Stile definieren

Die wahre Stärke des `listings`-Pakets liegt in der Möglichkeit, eigene Stilvorlagen zu erstellen. Mit `\lstdefinestyle` können Sie einmal definierte Formatierungen für alle Code-Beispiele in Ihrem Dokument verwenden.

2.1 Einen Stil definieren

Hier ist ein Beispiel für einen benutzerdefinierten Stil, der speziell für LaTeX-Code optimiert ist:

```
%Einstellung der Codeumgebung fuer LaTeX-Code-Beispiele
\lstdefinestyle{latex}{%
  language=TeX,%
  frame=single,%
  backgroundcolor=\color{yellow!20},%
  rulecolor=\color{lightgray!40},%
  breaklines=true,%
  basicstyle=\scriptsize\ttfamily,%
  keywordstyle=\bfseries\color{purple!40!black},%
  commentstyle=\itshape\color{green!40!black},%
  identifierstyle=\color{blue},%
  stringstyle=\color{black},%
  captionpos=b,%
  tabsize=2,%
  showspaces=false,%
  showstringspaces=false,%
  showtabs=false,%
  deletekeywords={document},%
  morekeywords={begin}%
}
```

2.2 Die wichtigsten Stil-Parameter

Die Anpassungsmöglichkeiten des `listings`-Pakets sind vielfältig. Hier sind die wichtigsten Parameter erklärt:

- `language=`: Bestimmt die Programmiersprache für die Syntaxhervorhebung

- `frame=`: Zeichnet einen Rahmen um den Code (`single`, `double`, `none`)
- `backgroundcolor=`: Setzt die Hintergrundfarbe des Code-Blocks
- `breaklines=true`: Ermöglicht automatische Zeilenumbrüche bei langen Zeilen
- `basicstyle=`: Definiert die Grundschriftart und -größe
- `keywordstyle=`: Formatierung für Schlüsselwörter der Programmiersprache
- `commentstyle=`: Formatierung für Kommentare
- `numbers=left/right`: Aktiviert Zeilennummerierung
- `tabsize=`: Bestimmt die Breite von Tabulatorzeichen

2.3 Weitere nützliche Stile

Hier sind Beispiele für spezialisierte Stile für andere Programmiersprachen:

```
% Stil fuer Python-Code
\lstdefinestyle{python}{%
  language=Python,%
  frame=single,%
  backgroundcolor=\color{blue!5},%
  breaklines=true,%
  basicstyle=\footnotesize\ttfamily,%
  keywordstyle=\bfseries\color{blue!70!black},%
  commentstyle=\itshape\color{green!60!black},%
  stringstyle=\color{red!70!black},%
  numbers=left,%
  numberstyle=\tiny\color{gray},%
  showstringspaces=false%
}
```

```
% Stil fuer SQL-Code
\lstdefinestyle{sql}{%
language=SQL,%
frame=single,%
backgroundcolor=\color{green!5},%
breaklines=true,%
basicstyle=\footnotesize\ttfamily,%
keywordstyle=\bfseries\color{purple!70!black},%
commentstyle=\itshape\color{gray!70!black},%
morekeywords={SELECT,FROM,WHERE,INSERT,UPDATE,DELETE},%
showstringspaces=false%
}
```

3 Verschiedene Programmiersprachen

Das `listings`-Paket zeigt seine Stärke besonders in der Vielseitigkeit der unterstützten Sprachen. Hier sind Beispiele für verschiedene gängige Programmiersprachen. Beachten Sie, dass in den folgenden Beispielen auf die explizite Darstellung der `lstlisting`-Umgebung selbst verzichtet wird, um technische Komplikationen zu vermeiden. Die Verwendung erfolgt jeweils durch `\begin{lstlisting}[language=Sprache]` und dem entsprechenden `\end{lstlisting}`.

3.1 Java-Code

Für Java-Programme wird `language=Java` verwendet:

```
public class Rechner {
    public static void main(String[] args) {
        int a = 15;
        int b = 25;
        int summe = a + b;
        System.out.println("Summe: " + summe);
    }
}
```

3.2 HTML-Code

Für Webseiten-Markup wird `language=HTML` verwendet:

```
<!DOCTYPE html>
<html>
<head>
    <title>Meine erste Webseite</title>
</head>
<body>
    <h1>Willkommen!</h1>
    <p>Dies ist ein Beispiel-Paragraph.</p>
    <ul>
        <li>Erstes Listenelement</li>
        <li>Zweites Listenelement</li>
    </ul>
</body>
</html>
```

3.3 SQL-Code

Für Datenbankabfragen wird `language=SQL` verwendet:

```
SELECT name, alter, klasse
FROM schueler
WHERE alter >= 16
ORDER BY name ASC;

INSERT INTO schueler (name, alter, klasse)
VALUES ('Max Mustermann', 17, '11a');
```

3.4 Python-Code

Für Python-Skripte wird `language=Python` verwendet:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Berechnung der ersten 10 Fibonacci-Zahlen
for i in range(10):
    print(f"F({i}) = {fibonacci(i)}")
```

3.5 C-Code

Für C-Programme wird `language=C` verwendet:

```
#include <stdio.h>

int main() {
    int a = 15;
    int b = 25;
    int summe = a + b;
    printf("Summe: %d\n", summe);
    return 0;
}
```

4 Weitere nützliche Funktionen

4.1 Code aus Dateien einbinden

Neben der direkten Eingabe können Sie auch Code aus externen Dateien einbinden. Dies ist besonders praktisch für längere Programme:
Der Befehl `\lstinputlisting[language=Python]{dateiname.py}` bindet eine Python-Datei vollständig ein und wendet automatisch die entsprechende Syntaxhervorhebung an.

4.2 Beschriftungen für Code-Listings

Code-Listings können wie Abbildungen oder Tabellen mit Überschriften und Referenzen versehen werden:

```
\begin{lstlisting}[language=Java, caption=Einfaches Hello-World-Programm, label=lst:hello, captionpos=b]
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
\end{lstlisting}
```

Ausgabe

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Quellcode 1: Einfaches Hello-World-Programm

Die Beschriftung wird mit `caption=Text` hinzugefügt. Mit `captionpos=b` erscheint sie unterhalb des Code-Blocks, mit `captionpos=t` oberhalb (Standard). Mit `label=` können Sie dem Listing eine Referenz zuweisen, auf die Sie später mit `\ref{labelname}` verweisen können.

5 Zusammenfassung

Das `listings`-Paket ist ein mächtiges Werkzeug für die professionelle Darstellung von Quellcode in LaTeX-Dokumenten. Mit wenigen Grundbefehlen können Sie Code verschiedenster Programmiersprachen ansprechend formatieren. Die Möglichkeit, eigene Stilvorlagen zu definieren, ermöglicht es, konsistente und an Ihre Bedürfnisse angepasste Code-Darstellungen zu erstellen.
Die wichtigsten Vorteile auf einen Blick: Das Paket unterstützt über 60 Programmiersprachen mit automatischer Syntaxhervorhebung. Durch benutzerdefinierte Stile können Sie einheitliche Formatierungen für alle Ihre Unterrichtsmaterialien schaffen. Erweiterte Funktionen wie Zeilennummerierung, Captions und die Einbindung externer Dateien bieten maximale Flexibilität. Die einfache Integration in bestehende LaTeX-Workflows spart Zeit bei der Dokumenterstellung.
Für spezifische Anpassungen und erweiterte Funktionen empfiehlt sich ein Blick in die umfassende Paketdokumentation, die über `texdoc listings` auf der Kommandozeile oder online verfügbar ist. Mit den hier vorgestellten Grundlagen sind Sie jedoch bereits bestens gerüstet, um professionelle Code-Darstellungen für Ihren MINT-Unterricht zu erstellen und dabei von der Einfachheit und Flexibilität des `listings`-Pakets zu profitieren.